Rejoignez la séance

Scannez le QR code ou entrez le code sur votre téléphone.

COURS

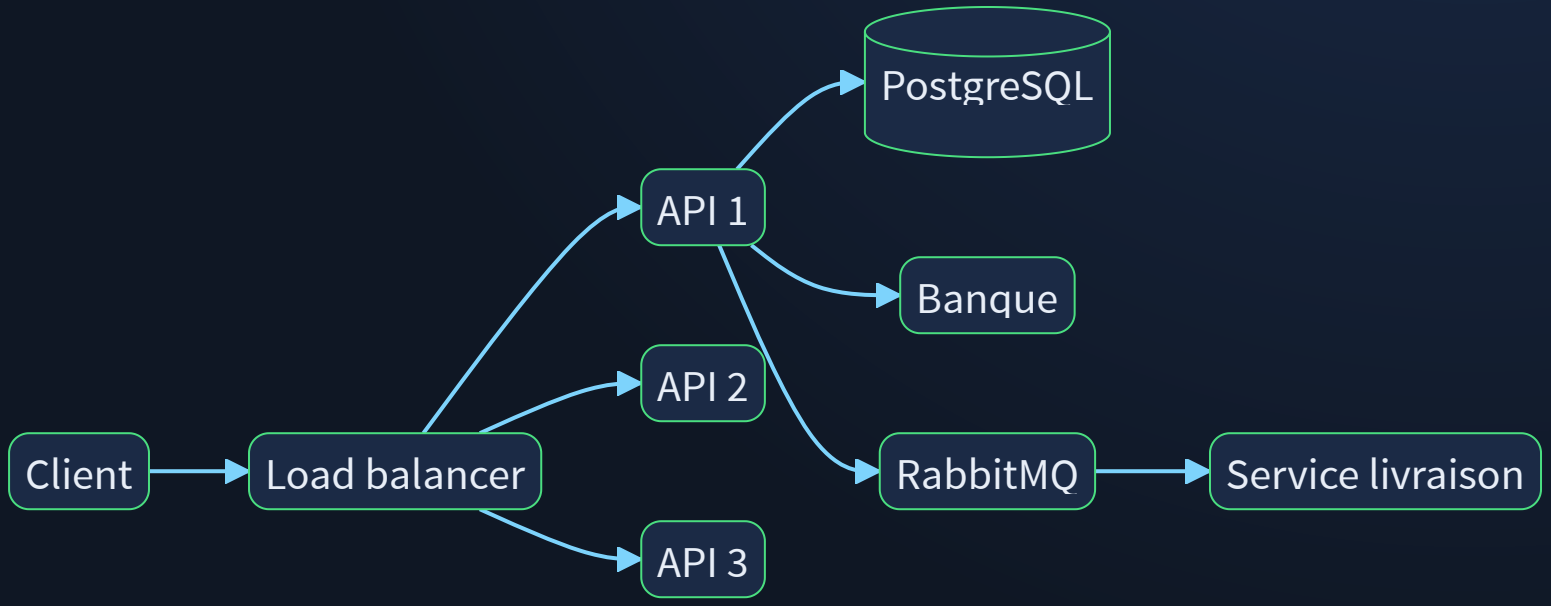
Observabilité

Comprendre ce qui se passe en production

Speaker notes

Fil rouge : FraisDirect, après le cours middleware. Elle a maintenant un load balancer, plusieurs instances, RabbitMQ et un service de livraison. Elle tourne en production, et un jour, elle est lente.

FraisDirect en production



Speaker notes

L'architecture à la fin du cours middleware, simplifiée : seule l'API 1 montre ses flèches, mais les trois font pareil.

Vendredi, 19 h

1. Les clients se plaignent : une commande prend 10 secondes
2. Le tableau de bord : tous les serveurs sont verts
3. Les logs : 3 instances, 2 millions de lignes
4. Par où commencer ?

Speaker notes

Demander à la salle ce qu'ils feraient. On garde la question en tête pendant tout le cours : on y répond à la fin, avec une trace.

Au programme

1. Monitoring, alerting, logs, **observabilité** : qui fait quoi
2. Bien écrire ses **logs**
3. Mesurer avec des **métriques**
4. Suivre une requête avec des **traces**

Speaker notes

Les trois derniers points sont les trois signaux d'OpenTelemetry.

ACTE 1

Qui fait quoi ?

Monitoring, alerting, logs, observabilité

Speaker notes

Quatre mots qu'on mélange souvent. Chacun répond à une question différente.

Le monitoring

Vérifier à intervalle régulier une assertion

Un test unitaire, mais en production

- ⊕ Détecte les pannes qu'on sait prévoir : serveur arrêté, disque plein, réseau coupé
- ⊖ Ne détecte pas les pannes inconnues
- ⊖ N'aide pas à résoudre un bug

Speaker notes

Le monitoring répond à « est-ce que ça marche ? ». Vendredi 19 h, tous les tests du monitoring passent : les serveurs répondent, simplement lentement. Le réseau est le cas le plus piégeux : le serveur de monitoring peut être le seul à ne plus le voir.

L'alerting

- Prévenir quelqu'un quand une panne est détectée
- Permet de réagir vite
- Ne dit pas pourquoi

Speaker notes

Un mail, un SMS, un message à l'astreinte. L'alerting s'appuie sur le monitoring ou sur les métriques.

Les logs

- ⊕ Suivre une action en particulier : la commande 42
- ⊖ Pas de vision globale : combien de commandes lentes ? depuis quand ?

Speaker notes

Les logs répondent à « que s'est-il passé pour cette requête ? », mais pas à « que se passe-t-il en général ? ».

L'observabilité

Tout le reste

- ⊕ Comprendre une panne, même inconnue
- ⊕ Faciliter sa résolution
- ⊕ Être sûr qu'on en est sorti
- ⊖ Ne remplace pas le monitoring, les logs et l'alerting

Speaker notes

Le monitoring répond aux questions qu'on a prévues. L'observabilité permet de poser des questions qu'on n'avait pas prévues : « les commandes sont lentes, mais seulement celles payées par carte, seulement depuis 18 h ».

En résumé

Question	Outil
Est-ce que ça marche ?	Monitoring
Qui doit réagir ?	Alerting
Que s'est-il passé pour cette requête ?	Logs
Pourquoi ça ne marche pas ?	Observabilité

Speaker notes

Faire relire le tableau à voix haute, puis question.

QUESTION

Le site ne répond plus à 3 h du matin. Envoyer un SMS à la personne d'astreinte, c'est le rôle…

A. du monitoring

B. de l'alerting

C. des logs

D. de l'observabilité

Speaker notes

Réponse B. Le monitoring a détecté la panne, l'alerting prévient. Les deux vont ensemble mais ne font pas la même chose.

Pourquoi maintenant ?

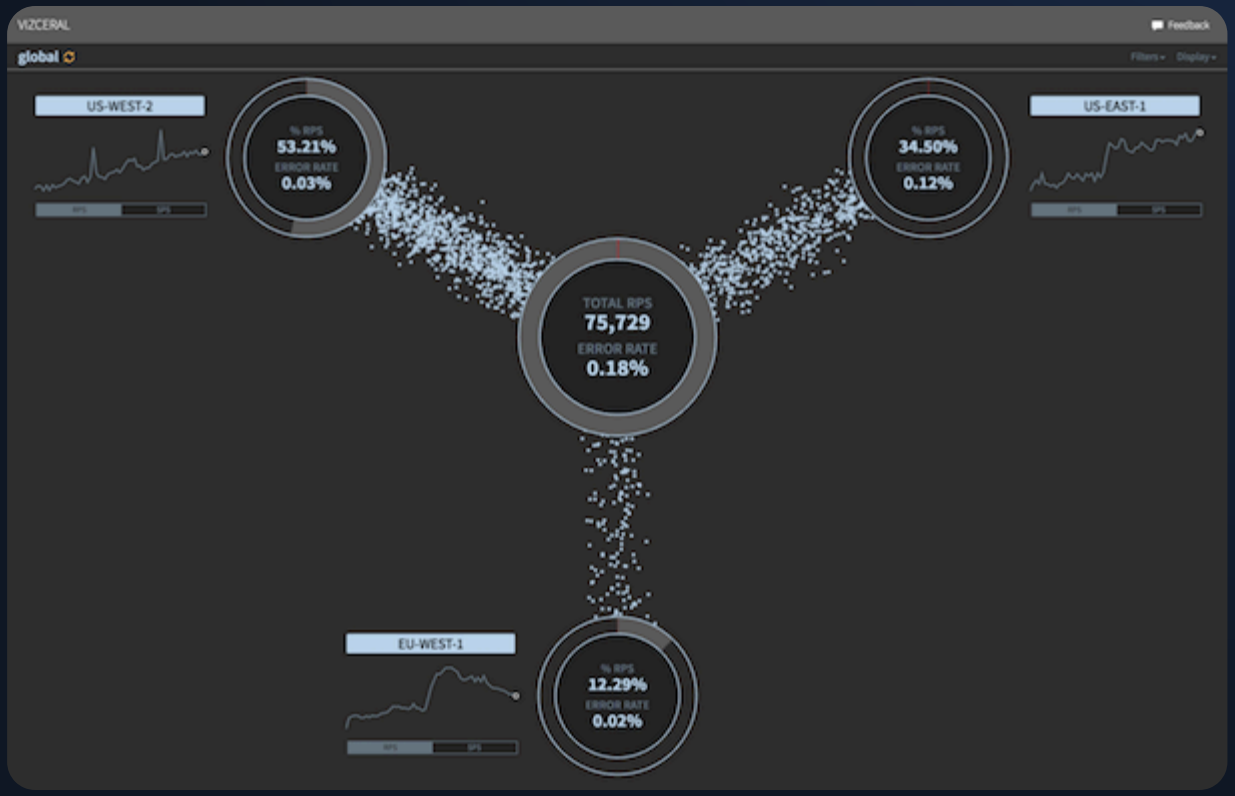
- Des systèmes de plus en plus complexes : FraisDirect a déjà 6 composants
- Des disponibilités de plus en plus hautes

Disponibilité	Panne autorisée par an
99 %	3,65 jours
99,9 %	8 h 45
99,99 %	52 minutes

Speaker notes

À 99,99 %, on a 52 minutes par an pour trouver et corriger. Impossible si on passe une heure à lire des logs pour comprendre.

Un exemple : Vizceral



Speaker notes

L'outil de Netflix : chaque point est une requête, en temps réel. Cas d'usage : après une maintenance, un datacenter n'a pas été rallumé. Aucune alerte, puisque rien n'est en panne, mais on voit tout de suite qu'une branche ne reçoit plus de trafic.

OpenTelemetry

Un standard pour produire les signaux

Signal	Répond à
Logs	Que s'est-il passé ?
Métriques	Combien, à quelle vitesse ?
Traces	Par où est passée la requête ?
Baggage	Quel contexte transporter avec elle ?

Speaker notes

OpenTelemetry (OTel) définit le format et fournit les librairies. On choisit ensuite où envoyer les signaux. Le baggage est plus rare : des données métier qui voyagent avec la requête, comme l'identifiant du client. On voit les trois premiers, un par acte.

ACTE 2

Les logs

Écrire des logs qu'on pourra relire

Speaker notes

Tout le monde écrit des logs. Peu de gens écrivent des logs utiles à 19 h un vendredi.

Le bon niveau

Niveau	Exemple FraisDirect
`ERROR`	La base ne répond plus, la commande est perdue
`WARN`	La banque a répondu en erreur, on réessaie
`INFO`	Commande 42 créée
`DEBUG`	Détail du calcul des frais de livraison
`TRACE`	Chaque ligne du panier

Speaker notes

ERROR : quelqu'un doit agir. WARN : anormal, mais l'application s'en sort. INFO : ce qui change l'état du système. En production on garde INFO, on active DEBUG temporairement pour enquêter.

QUESTION

Le paiement d'une commande échoue définitivement : la banque refuse après trois essais. Quel niveau ?

A. `DEBUG`

B. `INFO`

C. `WARN`

D. `ERROR`

Speaker notes

Réponse D si la commande est perdue et que quelqu'un doit agir. Discussion intéressante si certains répondent WARN : une carte refusée pour solde insuffisant est un cas métier normal, pas une erreur. Tout dépend de qui doit réagir.

Un log utile

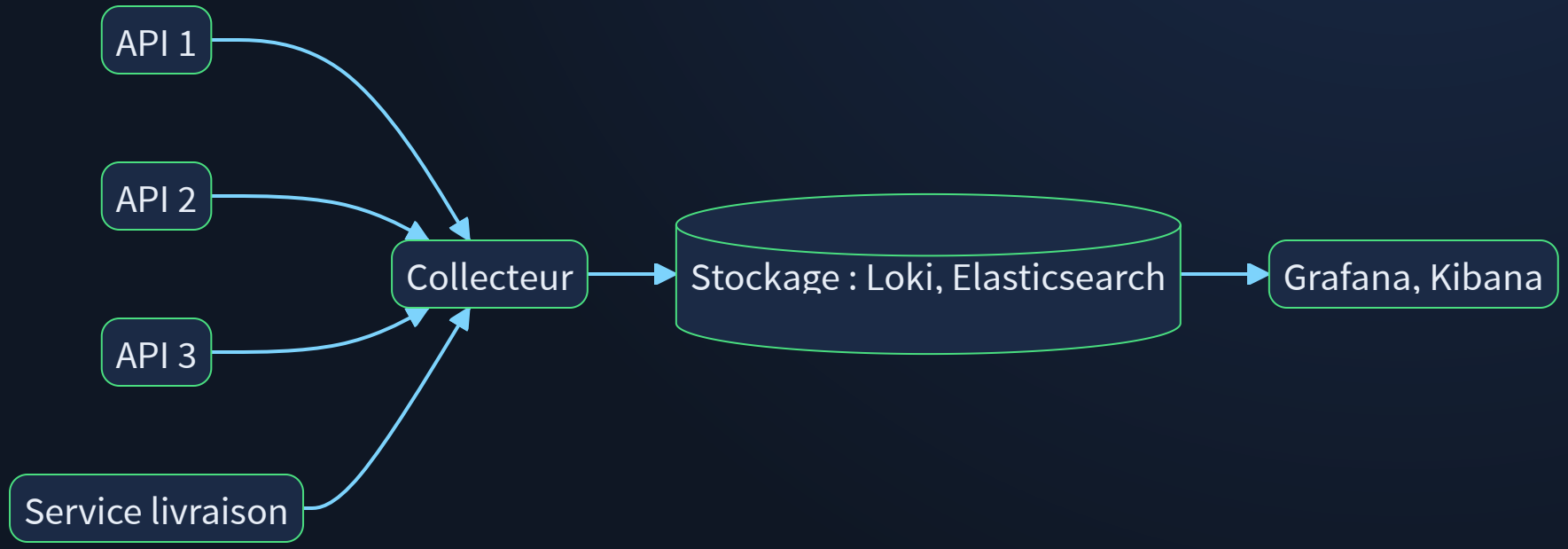
```
1 log.info("Commande créée");
2
3 log.info("Commande {} créée pour le client {}, {} centimes",
4         commande.id(), commande.client(), commande.montant());
```

- Toujours l'identifiant de ce qu'on manipule
- Logguer les **écritures** : création, modification, suppression
- Jamais de mot de passe, de token ou de numéro de carte

Speaker notes

Le premier log ne sert à rien le vendredi soir : quelle commande ? Les `{}` évitent de construire la chaîne si le niveau est désactivé. Les écritures sont ce qu'on cherche en premier quand on enquête : qui a changé quoi, et quand. Le dernier point rejoint le cours sécurité.

Agréger les logs



Un seul endroit pour chercher, au lieu de quatre serveurs

Speaker notes

Avec le load balancer, la commande 42 peut avoir été traitée par n'importe quelle instance. Sans agrégation, il faut se connecter à chaque serveur. Avec, on cherche « commande 42 » une seule fois.

ACTE 3

Les métriques

Des chiffres sur l'application, dans le temps

Speaker notes

Les logs racontent une requête. Les métriques donnent la vision globale qui leur manque : combien de commandes lentes, depuis quand.

C'est quoi une métrique ?

Une mesure de quelque chose dans l'application

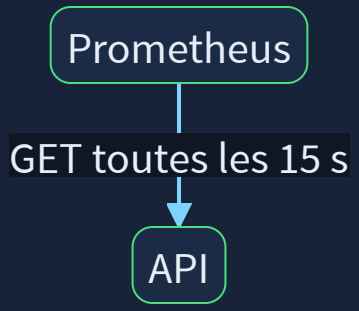
- Un nombre d'actions : commandes passées
- Une durée : temps d'une commande
- Une ressource réservée : connexions à la base

Speaker notes

Une métrique est un nombre, mesuré régulièrement. On ne regarde jamais une valeur seule, mais une courbe.

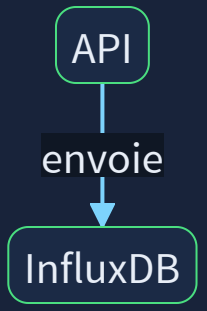
Récupérer les métriques

Scraping



Le système interroge l'application

Push



L'application pousse ses mesures

Speaker notes

Prometheus est le plus courant, et c'est celui du projet. En scraping, une application qui ne répond plus ne donne plus de métriques : c'est déjà une information.

Ajouter des métriques

- ⊖ À la main, avec un endpoint maison : mauvaise idée
- ⊕ Avec **Micrometer** : une façade, comme SLF4J pour les logs
- ⊕ Compatible avec Prometheus, InfluxDB, Datadog…

Speaker notes

On écrit le code de mesure une fois avec Micrometer. Changer de Prometheus vers autre chose ne demande qu'une dépendance différente.

POUR LE PROJET

Étape 1 · Les dépendances

`build.gradle`

```
implementation "org.springframework.boot:spring-boot-starter-actuator"
implementation "io.micrometer:micrometer-registry-prometheus"
```

Speaker notes

Pas de version à préciser : Spring Boot gère celle de Micrometer. Actuator fournit les endpoints, le registre Prometheus formate les métriques.

POUR LE PROJET

Étape 2 · Exposer l'endpoint

`application.yaml`

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
```

Speaker notes

C'est l'endpoint prometheus qu'il faut exposer, pas seulement metrics : c'est lui que Prometheus vient lire. Attention, il ne doit pas être accessible depuis Internet.

POUR LE PROJET

Étape 3 · Vérifier

`GET /actuator/prometheus`

```
# TYPE http_server_requests_seconds summary
http_server_requests_seconds_count{method="POST",uri="/commandes"} 1532
http_server_requests_seconds_sum{method="POST",uri="/commandes"} 412.7
# TYPE jvm_memory_used_bytes gauge
jvm_memory_used_bytes{area="heap"} 1.43E8
```

Speaker notes

Spring Boot fournit déjà beaucoup de métriques sans rien coder : requêtes HTTP, mémoire, pool de connexions. Les étudiants peuvent le vérifier tout de suite avec curl.

Les types de mesure

Type	Exemple FraisDirect
Compteur (Counter)	Commandes passées
Durée (Timer)	Temps de passage d'une commande
Jauge (Gauge)	Messages en attente dans RabbitMQ
Distribution (Distribution summary)	Montant des paniers

Speaker notes

Un compteur ne fait que monter. Une jauge monte et descend. Un timer et une distribution donnent le nombre, la somme, le maximum, et des percentiles si on les active.

QUESTION

On veut suivre le nombre de livreurs disponibles en ce moment. Quel type de mesure ?

A. Compteur

B. Durée

C. Jauge

D. Distribution

Speaker notes

Réponse C : la valeur monte et descend. Un compteur ne peut que monter.

Mesurer une commande : mauvaise façon

```java
public void passerCommande(Commande commande) {
    commandesPassees.increment();
    chrono.record(() -> {
        try {
            verifierStock(commande);
            payer(commande);
            publierEvenement(commande);
        } catch (RuntimeException e) {
            commandesEchouees.increment();
            throw e;
        }
    });
}
```

Speaker notes

Trois lignes de métier noyées dans le code de mesure. Et il faut refaire la même chose dans chaque méthode qu'on veut mesurer.

Mieux : séparer métier et mesure

```java
public interface CommandeService {
    void passerCommande(Commande commande);
}

public class CommandeServiceImpl implements CommandeService {
    public void passerCommande(Commande commande) {
        verifierStock(commande);
        payer(commande);
        publierEvenement(commande);
    }
}
```

Speaker notes

Étape 1 : une interface, et une implémentation qui ne contient que le métier. Lien avec le cours bonnes pratiques : une classe, une seule raison de changer.

Mieux : la délégation

```java
public class CommandeServiceMesure implements CommandeService {
    private final CommandeService delegue;

    public void passerCommande(Commande commande) {
        commandesPassees.increment();
        chrono.record(() -> {
            try {
                delegue.passerCommande(commande);
            } catch (RuntimeException e) {
                commandesEchouees.increment();
                throw e;
            }
        });
    }
}
```

Speaker notes

Étape 2 : une deuxième implémentation qui mesure et passe la main à la vraie. C'est de la composition, le patron « décorateur ». Le métier ne sait pas qu'il est mesuré.

Mieux : un aspect

```java
@Aspect
@Component
class CommandeMetriques {
    @Around("execution(* *..CommandeService.passerCommande(..))")
    Object mesurer(ProceedingJoinPoint appel) throws Throwable {
        commandesPassees.increment();
        Timer.Sample chrono = Timer.start(registry);
        try {
            return appel.proceed();
        } catch (Throwable e) {
            commandesEchouees.increment();
            throw e;
        } finally {
            chrono.stop(registry.timer("commande.duree"));
        }
    }
}
```

Speaker notes

Étape 3 : Spring exécute ce code autour de la méthode, sans interface ni classe en plus. Il faut spring-boot-starter-aop. L'expression @Around désigne les méthodes concernées.

Encore plus simple : `@Timed`

```java
@Timed("commande.duree")
public void passerCommande(Commande commande) {
    verifierStock(commande);
    payer(commande);
    publierEvenement(commande);
}
```

Micrometer fournit l'aspect tout fait

Speaker notes

@Timed et @Counted fonctionnent avec un bean TimedAspect (et CountedAspect) à déclarer. Pour les cas simples, c'est la solution à préférer. L'aspect écrit à la main reste utile quand on veut une logique particulière.

Tagguer les métriques

```
registry.counter("http.appels", "path", path).increment();
```

- Le nombre d'appels, c'est bien
- Le nombre d'appels **par endpoint**, c'est mieux
- Jamais de valeur sans limite : pas d'identifiant de client en tag

Speaker notes

Un tag ajoute du contexte, et permet de filtrer et de regrouper. Mais chaque valeur de tag crée une nouvelle série dans Prometheus : avec un identifiant de client, on crée un million de séries et Prometheus tombe.

Tagguer toutes les métriques

```
registry.config().commonTags("instance", nomDuServeur);
```

Avec le load balancer : quelle instance est lente ?

Speaker notes

Un tag commun s'ajoute à toutes les métriques de l'application. Avec Spring Boot, on peut aussi l'écrire dans application.yaml : management.metrics.tags.instance. Vendredi 19 h, on verrait tout de suite si une seule instance est lente, ou les trois.

ACTE 4

Les traces

Suivre une requête de bout en bout

Speaker notes

Les métriques disent que les commandes sont lentes, sur les trois instances. Mais où se perd le temps ? Base, banque, RabbitMQ ? C'est le rôle des traces.

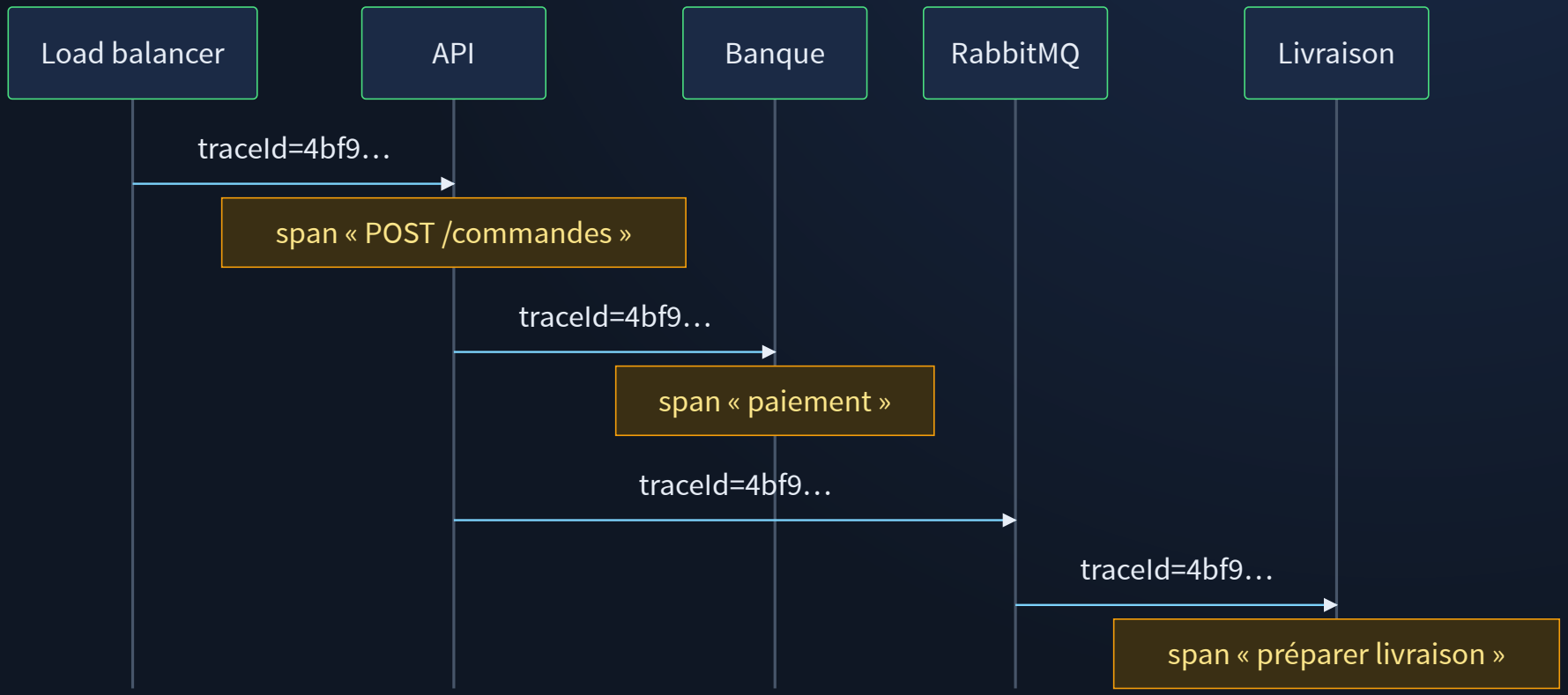
Le principe

1. Quand une requête arrive, on lui donne un **traceId**
2. On fait suivre le traceId pendant toute la requête, d'un service à l'autre
3. On découpe le traitement en **spans** : une étape, avec un début et une fin

Speaker notes

Une trace, c'est l'arbre de tous les spans d'une même requête.

Le traceId voyage



Speaker notes

Le traceId passe dans un en-tête HTTP (traceparent) ou dans les en-têtes du message RabbitMQ. Chaque service crée ses spans avec le même traceId.

Une trace

`POST /commandes` : 1 240 ms



Speaker notes

Chaque barre est un span, placé dans le temps. Faire chercher où passe le temps : la banque, 1 030 ms sur 1 240. Ce n'est ni notre code, ni notre base.

QUESTION

Vendredi 19 h : la trace montre 1 030 ms sur 1 240 passées à attendre la banque. Que faire en premier ?

A. Ajouter une quatrième instance de l'API

B. Ajouter un index en base

C. Regarder du côté de la banque

D. Redémarrer RabbitMQ

Speaker notes

Réponse C. A et B sont les réflexes classiques, et ne changeraient rien : le temps est passé à attendre la banque. Sans trace, on aurait pu passer la soirée à optimiser la base.

Les traces, à quoi ça sert ?

- Relier les lignes de log d'une même requête
- Voir les communications entre les services
- Corréler les logs et les métriques
- Trouver les points de lenteur d'une requête

Speaker notes

Le premier point est le plus utile au quotidien : on cherche le traceId dans les logs agrégés, et on obtient toutes les lignes de la requête, sur tous les services.

Avec Spring Boot

```
implementation "io.micrometer:micrometer-tracing-bridge-otel"
implementation "io.opentelemetry:opentelemetry-exporter-otlp"
```

Le traceId apparaît tout seul dans les logs

```
INFO [api,4bf92f3577b34da6,00f067aa0ba902b7] Commande 42 créée
```

Speaker notes

Spring Boot propage le traceId dans RestClient, WebClient et RabbitTemplate. Les traces partent vers un collecteur OpenTelemetry, puis vers Jaeger, Tempo ou Zipkin. Dans le log : le nom de l'application, le traceId, puis le spanId.

À retenir

- Le monitoring dit si ça marche, l'observabilité dit pourquoi
- Des logs avec le bon niveau et des identifiants, agrégés
- Des métriques avec Micrometer, séparées du code métier
- Des traces pour suivre une requête entre les services

Speaker notes

Retour au vendredi 19 h : les métriques montrent que les commandes sont lentes sur les trois instances, la trace montre que c'est la banque, les logs du traceId donnent le détail de l'erreur. Trois signaux, une réponse en quelques minutes.